Practical 1 – Physics – I Miss My Cats

Overview

The goal of the project was to create a video game that implements a basic physic engine and collision system. The game I created is currently called Cat Game and is a 2D platformer where the player can switch between different cat characters with different special abilities. I created a sophisticated Tile Map implementation to represent the levels of the platformer. Additionally, the player stays stationary on the screen, so the physics system works based on offsets generated by the player's location on the Tile Map, not the screen. The collision system is another core feature and works on the assumption that all entities have hitboxes and if those hitboxes overlap the system derives a way to handle that overlap.

Design

Cat Game is made up of three main core game mechanics that all work together to create the final experience. These include a sophisticated Tile Map implementation, an offset-based physics system, and a hitbox collision system. First and foremost, the Tile Map makes up the backbones of the game and is how levels are created. It stores information like win conditions, lose conditions, behavior changing surfaces, and entity spawn points. Additionally, lots of other information is compared to the Tile Map like the player's world location (not screen location) and the locations of other particles in the world. This information is used by the point-mass physics system to calculate how the screen is rendered based on the forces enacted on the player. Currently the physics system only tracks player-related physics. The collision system help make sure that the physics engine follows the rules of the world like that player can't go through solid walls or climb smooth surfaces. The collision system works closely with the Tile Map because

each tile stores behavior affecting information which tells the collision system how to handle collisions.

Technical Description

Each of the main game mechanics presented their own set of inherent challenges. The Tile Map was the first challenge in this practical. The Tile Map class takes a string that represents a file name for a level files. Level files are txt files (Figure 1) that contain a grid of integers. Each integer represents a different kind of tile that is implemented in the game. The constructor reads the file and generates all the different tiles in a matrix. However, the most challenging part of the tile map was the draw() function.

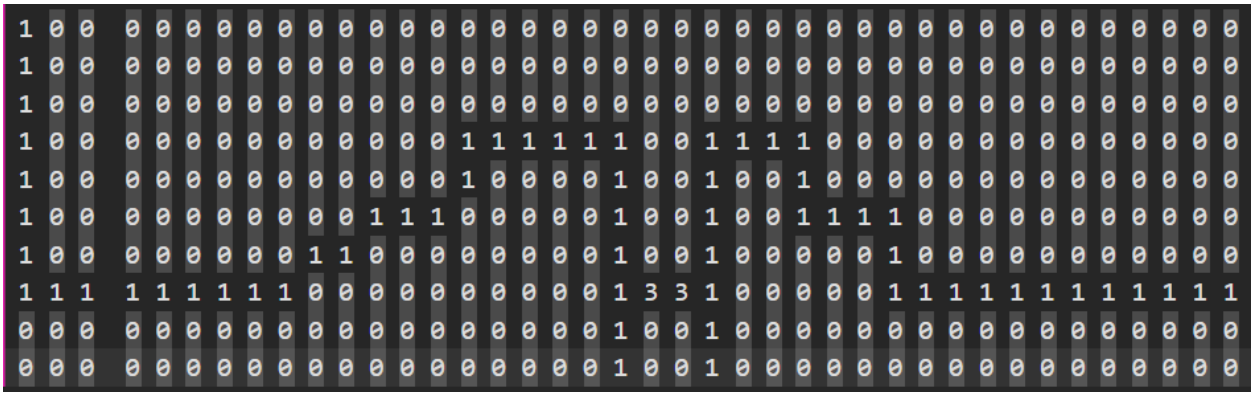


Figure 1

Originally, the draw function only took two inputs, offset x and offset y, which it used to calculate the range of the tiles that should be displayed and their position on the screen. This resulted in a blank screen as the offset did not track where the player is on the map, only how much they have moved from their starting position. The final form of draw takes both the offset and the Tile Map location of the player to draw tiles. The tiles that appear on the screen are calculated based on the player's location on the Tile Map and the screen position of each tile is calculated by the offset and its Tile Map location. This draw method is optimized as well. It

would be simpler to loop through every tile and draw everything based on the offset, but that would be a waste of time and resources. This method only draws the tiles that appear on the screen, rather than the entire map.

Another challenge was the point-based physics system because the player character, which all forces in this game are applied to, does not move on the screen. The player character was made up of particles that created a hitbox. Any forces applied to the character were applied to every particle that made up the character equally. The character particles had to be integrated in their own method that calculated the new position without changing the player's position. This data was used to calculate the player's new Tile Map location which is used to calculate the relative location of everything else on the screen. Implementing the force of Gravity created challenges in the collision system which will be explored later in this paper. Implementing springs however was purely a physics and logical challenge. Since the player does not actually move on the screen, the spring does not work as expected since it cannot force the target to move. To resolve this, the Jump Spring class creates a spring with two particles, an anchor particle that's not affected by the spring force and a target particle that is affected by the force. The player's particles were the target and a new list of jumping particles with the anchor with a static location relative to the Tile Map and a dynamic location on the screen. The offset calculated from the player's particles was applied to change the location of the jumping particles instead, thus allowing the spring to be properly represented, without the target moving.

The collision system works on the assumption that the world is made of hitboxes in the form of entities, like the player character or things that move independently of the Tile Map, and static boxes, tiles on the Tile Map. While this distinction does not matter in terms calculating collisions, it help with handling collisions in the dynamic objects. The collision takes two entities

as input, each entity represented by a PVector, a width, and a height (to make a hitbox), these entities are turned into rectangles that are then compared using built in methods to find an intersection. Intersections are handled by dynamic entities. In the player class, if a collision is detected, the method correctCollision is called which returns a PVector that is used to adjust the player's position in a one-dimensional direction. The calculation for the correction is based on the rectangles above and simple calls the built-in intersection method on rectangles and originally gave the new PVector the values of the width(x) and height(y) after being modified to considering the position of the correction (left, right, above, below). However, this two-dimensional changed created a bug where small collisions could cause a chain reaction the shot the player across the screen or through entities. To fix this, correctCollision only passes the larger number to the PVector and zero to the other dimension. The one-dimensional change is simpler and has less potential for bugs. Also, the implementation of Gravity created a problem where the player would be forced through solid problems. To fix this, a behavior was added whenever the player is touching a platform called walking. If the player is walking (above and touching at least one platform), Gravity does not affect the player.

Discussion and Conclusion

I feel like I did well on this practical. I was able to implement a physics system with Gravity and Springs that worked counter intuitively to the in-class examples as a result of the Tile Map. I am proud of my implementation of the Tile Map and its universal contributions to every other element of my project. I spent hours designing the Tiles and Mappings logic, creating levels (I have more on my iPad to be added), drawing the pixel graphics by hand, and using java based logic to make it work generically. I could not imagine building a game without it, and the

accessibility of level addition and design of the Tile and Entity system to this game makes it exceptionally easy to expand on this game in the future.

Moving forward, I would like to add more entities to the game, both hostile and non-hostile, that move independently of the Tile Map. I wanted to do this for this project, but I did not have the time. The hurdle I would have to get over for this to work is to figure out how to communicate the spawn position and information set in the Tile Map to the main function. I would also like to add animations to the main characters. I had so much fun drawing pixel art version of my cats and the app I used allows you to draw multi-frame pixel art that you can turn into GIFs which can be used as images in Processing. Of course, adding this will mean that I will have to increase the memory allowances of the Processing IDE which will make usage more complicated for other users, but cute cats are worth risking it for. On the note of graphics, I would like to create more interesting tiles. Most of the static tiles are stand-ins until I figure something else out. My vision was that the cats would be venturing through the house, so ideally the tiles would be related to that (like couches, counters, shelves, etc.). Moving on from graphics, I would have liked to implement more forces, but I could not think of anything relating to how the cats moved and the world. I think in the future more forces will come with adding new entities and behaviors to the game (like baited catnip that the cats are attracted to). I would also like to work jumping behavior because sometimes the cat does not jump right away or at all when the space bar is pressed.

Overall, I am happy with the work I have done on the practical and I feel like my passion for this project really shows in my code and the pixel cat drawings I spent hours on. I faced many challenges in implementing the physics in unintuitive ways, figuring out how to display the tiles correctly in relation to the player, and handling collision quickly and correctly. However, I

overcame these challenges and was able to create a product that works fairly seamlessly. To reiterate, I am very happy with my Tile Map implementation and its potential for future level creation, and my player/cat implementation. When I was coming up with the concept of this game, I was not sure that switching characters would be something that I could implement and would work in the context of the game, but after implementing it I feel like it adds to the experience of the game immensely. I am proud of my work on this practical and I look forward to working on it in the future.

References

Tile Maps Concept https://udn.realityripple.com/docs/Games/Techniques/Tilemaps

CS4303 Lecture Materials and Examples https://studres.cs.st-andrews.ac.uk/CS4303/Lectures/